

Verificação, Validação e Teste de Software (VVT) 2026.1

Week 1 - Day 2 - Conceitos básicos para a aplicação das atividades de VVT

Prof. Dr. Awdren de Lima Fontão



UNIVERSIDADE
FEDERAL DE
MATO GROSSO DO SUL

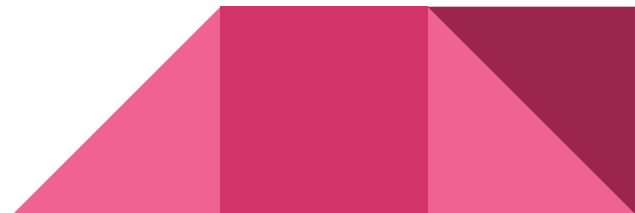


Aquecimento

Duplas - Dev/Tester

Rodadas de 2 min

- Desenvolvedor: apresente uma sequência de números que siga uma determinada lógica;
- O testador receberá somente a lógica para a geração do número.



Discussão

"O que aconteceu quando a interpretação foi diferente da intenção?"

"Se tivéssemos testado as instruções antes, os erros poderiam ter sido evitados?"

"Como isso se relaciona com a importância de testar software?"

"Na engenharia de software, quem garante que as instruções (código) funcionem corretamente?"



Refinamentos - DoR/DoD

Estória de Usuário

 **Título:** Criar um código secreto baseado em uma lógica definida.

Como um estudante de VVT,

Quero escrever um código secreto seguindo uma lógica clara,

Para que outro aluno possa interpretá-lo corretamente sem ambiguidade.



DoR

Antes de começar a atividade, as seguintes condições devem ser atendidas:

- ✓ A instrução da atividade está claramente definida e revisada.
- ✓ A lógica escolhida para o código secreto segue um padrão identificável.
- ✓ A equipe tem um tempo pré-definido para elaborar e testar a lógica antes de entregá-la.
- ✓ O código secreto tem exatamente **4 números** e deve vir acompanhado de uma explicação clara.
- ✓ Há um exemplo de aplicação da regra
- ✓ A lógica é descrita com operador/condição
(ex.: 'multiplica por 2', '+3', 'próximo primo')



Exemplo de instrução bem construída

Tarefa: Escreva um código secreto de 4 números baseado em uma lógica específica. Explique a lógica de forma que outra pessoa possa descobrir o padrão e continuar a sequência.

- ◆ **Formato esperado da resposta:**

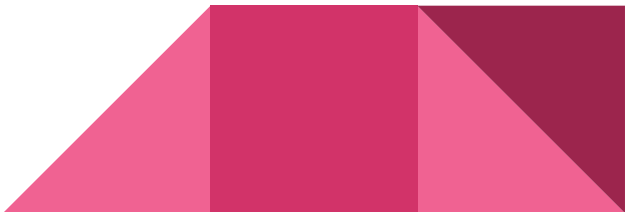
- **Código secreto:** 2, 4, 8, 16
- **Explicação:** Cada número é o dobro do anterior (multiplicado por 2).

- ◆ **Regras:**

- O código deve ser baseado em uma lógica matemática ou sequencial.
 - A explicação deve ser objetiva e clara.
 - O código e a explicação devem ser escritos sem dar diretamente a resposta.
- 

DoD

A atividade será considerada concluída com sucesso quando:

- ✓ O código contém exatamente 4 números.
 - ✓ A explicação da lógica está suficientemente clara para permitir que um colega continue a sequência corretamente.
 - ✓ Outro aluno consegue identificar o padrão sem precisar de informações adicionais.
 - ✓ A resposta foi revisada e não contém ambiguidades.
- 

Mas o que é um software ter qualidade?

Não falhar?

Dar respostas rápidas?

As duas coisas?

Nenhuma das duas?

Outras coisas?



Mudando temporariamente de assunto...

- O que indica que essa aula/disciplinacurso teve qualidade?
 - Todos os alunos serem aprovados com notas altas?
 - Todos virem pra aula?
 - Todos participarem das discussões?
 - As três coisas?
 - Nenhuma delas?
 - Outras coisas?



Mas o que é um software ter qualidade?

Não falhar?

Dar respostas rápidas?

As duas coisas?

Nenhuma das duas?

Outras coisas?

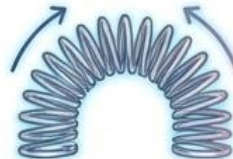
Atender às necessidades dos interessados pelo software desenvolvido, causando sua satisfação.



Qualidade de Software (1)

“Subárea da Engenharia de Software que trata de aspectos relacionados a processos, métodos, técnicas, ferramentas e ambientes que apoiam a obtenção e avaliação da qualidade do produto e do processo de software.”

Se refere a características mensuráveis: Coisas que podemos comparar com padrões conhecidos, como comprimento, cor, maleabilidade



Qualidade de Software (2)

Em software, ela está diretamente relacionada à:



Conformidade com requisitos:

Requisitos são especificados e espera-se que sejam atendidos.



Satisfação do cliente:

requisitos são especificados por pessoas para satisfazer outras pessoas.

- Uma boa especificação depende das escolhas feitas (clientes alvo).
- Pode haver problemas na especificação.

01

Dicionário

- Propriedade, atributo ou condição das coisas ou das pessoas capaz de distingui-las das outras e de lhes determinar a natureza.

02

NBR ISO 8402

- Totalidade das características de uma entidade que lhe confere a capacidade de satisfazer às necessidades implícitas e explícitas.

03

Crosby

- Conformidade com os requisitos, fazer certo da primeira vez.
- Zero defeitos.



Impacto Global da Atualização CrowdStrike (Julho 2024)

Uma **atualização defeituosa do software** de segurança da CrowdStrike causou **panes globais** em sistemas Windows, **afetando milhões de computadores** e **interrompendo operações** em diversos setores.



companhias
aéreas



bancos



serviços de
emergência

Após o incidente global de TI em julho de 2024, a Delta Air Lines entrou com uma ação judicial contra a CrowdStrike, alegando que a falha no software causou o cancelamento de milhares de voos e prejuízos superiores a US\$ 500 milhões.



TECNOLOGIA

Avião da Delta Air Lines decola do aeroporto de Sydney — Foto: Reuters

A empresa aérea norte-americana **Delta Air Lines** revelou nesta quarta-feira (31) que teve um prejuízo de US\$ 500 milhões (cerca de R\$ 2,83 bilhões) com o **apagão cibernético global do dia 19 de julho**.

A companhia confirmou que vai processar a CrowdStrike, responsável pelo problema, e também a **Microsoft**, dona do Windows, sistema operacional afetado pela pane, segundo informações da agência France Presse.

Perda de dinheiro... muito dinheiro

Sonda da NASA lançada para realizar o estudo das variáveis atmosféricas

Problema:



Foi destruído devido a um erro de navegação



Deveria efetuar sua inserção na órbita de Marte a uma altitude de 140 a 150 km da superfície.

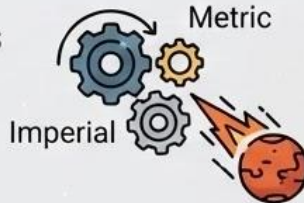


Devido a um “equivoco”, entrou a uma altitude de 57 km e foi destruída pela sua fricção com a atmosfera de Marte.

Causa:

- A equipe da terra usou medidas inglesas para calcular os parâmetros de inserção e enviou os dados a nave e esta apenas realizavam cálculos no sistema métrico.

Consequências: perda de US\$ 300 milhões



**Consequências: perda
de US\$ 300 milhões**

Pilares da Garantia da Qualidade de Software



Verificação (Requisitos de Segurança)

- Requisitos de segurança (hazards, intertravamentos, limites, condições perigosas) não foram formalizados/verificados de forma suficiente.
- Muitas vezes assumidos como garantidos por hardware/procedimento.



Validação (Uso Real)

- Falha em validar o sistema no contexto real de operação (interação humano-máquina).
- Inclui erros de uso previsíveis, pressa, repetição de comandos e comportamento do operador.



Teste (Concurrency/Race Conditions)

- Insuficiência de testes focados em condições de corrida, estados raros e sequências rápidas de entrada.
- Cenários que costumam escapar de testes 'felizes'.



Revisões/Inspeções

- Lacuna em revisão rigorosa de design/código com foco em segurança.
- Exemplos: estados, invariantes, tratamento de exceções, logs.

Perda de vida humana

Máquina de radioterapia controlada por computador (Therac-25)

⚠ Problema:	⚙ Causa:	⚰ Consequências:
• Doses indevidas de radiação emitidas	• Interface com usuário inapropriada • Documentação deficiente • Software reutilizado sem ser adaptado para o novo hardware • Software de sensores de falha com defeito	• Ao menos 5 mortes entre 1985 e 1987



[Home](#) > [Tecnologia](#)

| 964 Views | 3 Mins

Boeing 737 MAX: erro de software derruba dois aviões

quinta-feira, 29 de dezembro de 2022 em [Tecnologia](#)

Quem é o responsável?



Se um médico erra em uma cirurgia, ele é o responsável pelas consequências.



Se um engenheiro erra um cálculo em uma obra, ele é o responsável pelo prejuízo.



Se o motorista de um ônibus (piloto de avião) erra, ele é o responsável pelo acidente ocorrido.



Se um software deixa de funcionar em seu momento crucial, causando prejuízos ao seu usuário, quem seria o responsável?



Por que o engenheiro de software não deve ser o responsável por qualquer fracasso durante o desenvolvimento e após a entrega de um software?



Verificação & Validação (VV)

Qual a função de VV?

Assegurar que o software cumpra com suas especificações e atenda às necessidades dos usuários.



Verificação

- Visa assegurar que o **produto de uma fase** está de **acordo com os requisitos** especificados nas fases anteriores.
- Não busca assegurar que o produto está correto, mas averiguar se está de acordo com as especificações pré-estabelecidas.
- Em suma, a pergunta a ser feita é: **Estamos desenvolvendo o produto corretamente?**
 - Os artefatos construídos devem estar de acordo com a especificação do software.

A Verificação

é o processo de avaliar se os artefatos de software (requisitos, design, código) cumprem as condições impostas no início daquela fase.



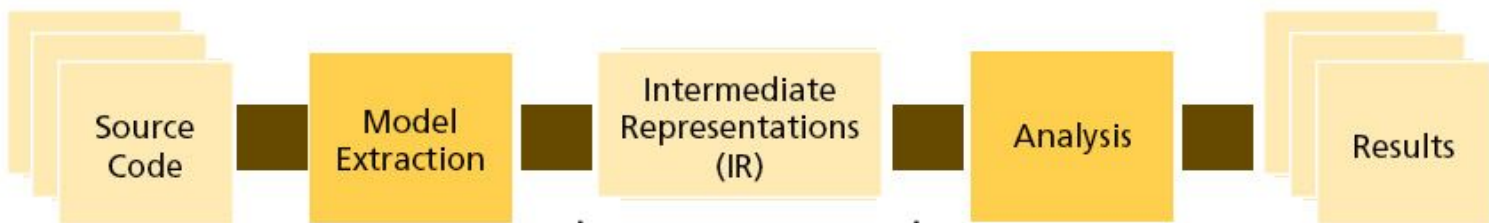
Foco: Conformidade técnica e fidelidade à especificação.



Pergunta-chave: "Estamos construindo a coisa da maneira que dissemos que construiríamos?"

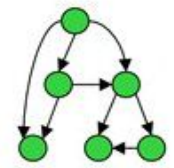
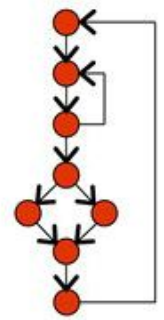
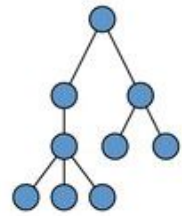


Exemplo na Realidade: Um Code Review...
...onde se checa se o código segue o padrão de arquitetura definido para o projeto. Se a arquitetura exige Clean Architecture e você entregou um script monolítico (mesmo que funcione), você falhou na verificação.



Na [REDACTED]

Name	Kind	Location
copy_item	function	item.c:25
item_cache	variable	item.c:10
color	parameter	palette.c:23
header.h	file	shapes.c



Validação

- Visa assegurar que o **produto de uma fase é apropriado e consistente** com as necessidades de clientes e usuários.
- A pergunta a ser feita é: **Estamos desenvolvendo o produto correto?**
 - O software deve atender às necessidades dos usuários.

O requisito do projeto era: *'recriar o Mew a partir do DNA'*.

A equipe entrega um organismo funcional, forte, estável... tudo parece ok em testes de laboratório...

Verificação aprovada: o artefato cumpre o que foi implementado (um clone/derivado do DNA).



Mas, quando o 'usuário' (a equipe/organização) olha para o resultado, percebe: não é o Mew; é outra coisa... o comportamento, as características e o 'propósito' são diferentes. Validação reprovada: o produto não resolve a necessidade original."

Revisão de Software

é um processo no qual um produto de software é examinado por pessoal do projeto, gerentes, usuários, clientes ou outras partes interessadas, com o objetivo de fornecer comentários e aprovações

Revisões de Software

- O que é?
 - Dois ou mais engenheiros verificam o produto de trabalho de um outro engenheiro, para encontrar defeitos e problemas.
- Objetivo?
 - Não é corrigir problemas e sim encontrá-los para que o desenvolvedor corrija depois.
- Quando?
 - O momento de fazer uma revisão é quando o engenheiro terminou o desenvolvimento do produto e corrigiu todos os defeitos óbvios.
 - Devem iniciar à medida que os primeiros artefatos forem produzidos.
 - Nesse ponto, o engenheiro precisa de ajuda para encontrar problemas remanescentes.

Revisões de Software

Pode ser vista como uma maneira de usar a diversidade de um grupo de pessoas para:

- Apontar melhorias necessárias ao produto;
- Confirmar as partes de um produto em que uma melhoria não é desejada ou não é necessária;
- Realizar um trabalho técnico com uma qualidade mais uniforme de forma a tornar este trabalho técnico mais administrável.

Revisões de Software

Revisões Técnicas



Revisões Técnicas

Avaliações técnicas em grupo de artefatos específicos.

Objetivo: Verificar conformidade com padrões e especificações, e a correção de modificações.

Inspeção



Inspeção

Técnica formal com foco na identificação e remoção de defeitos.

Obrigatório: Geração de lista de defeitos classificada e requisição de ações de correção.

Revisão de Código

- A revisão de código é uma prática amplamente adotada por diversas empresas de desenvolvimento de software, sendo considerada uma das principais estratégias para garantir a qualidade e a manutenibilidade dos sistemas.

```
1706 + voto_parlamentar = VotoParlamentar()
1707 + if(voto == 'sim'):
```

 **edwardoliveira** added a note a minute ago Owner ✎ ✕

Separar "if" do "(" com um espaço em branco, e remover os parênteses, nas linhas L#1707, L#1709, L#1711, L#1712, L#1713.

Add a line note

```
1708 + voto_parlamentar.voto = 'Sim'
1709 + elif(voto == 'nao'):
```

```
1710 + voto_parlamentar.voto = 'Não'
1711 + elif(voto == 'abstencao'):
```

```
1712 + voto_parlamentar.voto = 'Abstenção'
1713 + elif(voto == 'nao_votou'):
```

```
1714 + voto_parlamentar.voto = 'Não Votou'
1715 + voto_parlamentar.parlamentar_id = parlamentar_id
1716 + voto_parlamentar.votacao_id = votacao_id
```

Além de identificar e corrigir defeitos, ela promove a transferência de conhecimento entre os membros da equipe, evita a formação de "ilhas de conhecimento" e incentiva uma cultura de colaboração e troca de ideias.

Filtrar

Desenvolvimento em Kotlin para Android

Aprender Android e Kotlin do zero

Noções básicas do Android com o Compose

Aprender a linguagem Kotlin

Usar padrões comuns do Kotlin com o Android

Guia de estilo do Kotlin

Outros recursos

Kotlin para desenvolvedores Java do Android

Guias avançados do Kotlin

Como sua equipe pode adotar o Kotlin

Nomeação

Se um arquivo de origem contiver somente uma única classe de nível superior, o nome do arquivo precisará refletir o nome que diferencia maiúsculas de minúsculas e a extensão `.kt`. Caso contrário, se um arquivo de origem tiver várias declarações de nível superior, escolha um nome que descreva o conteúdo do arquivo, aplique o PascalCase (o camelCase será aceitável se o nome do arquivo estiver no plural) e anexe a extensão `.kt`.

```
// MyClass.kt
class MyClass { }
```

```
// Bar.kt
class Bar { }
fun Runnable.toBar(): Bar = // ..
```

```
// Map.kt
fun <T, O> Set<T>.map(func: (T) -> O): List<O> = // ..
fun <T, O> List<T>.map(func: (T) -> O): List<O> = // ..
```

```
// extensions.kt
fun MyClass.process() = // ..
fun MyResult.print() = // ..
```

Nesta página

Arquivos de origem

Nomeação

Caracteres especiais

Estrutura

Formatação

Chaves

Espaço em branco

Construções específicas

Nomeação

Documentação

<https://www.reviewboard.org/>

Algumas ferramentas

The platform for modern developers


Gerrit Code Review

Discuss code

Read old and new versions of files with syntax highlighting and colored differences. Discuss specific sections with others to make the right changes.

[gerrit / gerrit-server/src/main/java/com/google/gerrit/server/change/PatchSetInserter.java](#)

```

106 private PatchSet patchSet;
107 private ChangeMessage changeMessage;
108 private SshInfo sshInfo;
109 private ValidatePolicy validatePolicy = ValidatePolicy.GERRIT;
110 private boolean draft;
111 private boolean runHooks;

112 private boolean sendMail;
113 private Account.Id uploader;
114 private BatchRefUpdate batchRefUpdate;
115
116 @Inject
117 public PatchSetInserter(ChangeHooks hooks,
118 ReviewDb db,

```

```

110 private PatchSet patchSet;
111 private ChangeMessage changeMessage;
112 private SshInfo sshInfo;
113 private ValidatePolicy validatePolicy = ValidatePolicy.GERRIT;
114 private boolean draft;
115 private boolean runHooks = true;

```

 **Stefan Beller** Why do you move this out of the constructor? Initially I assumed it would be identical between the two constructors

```

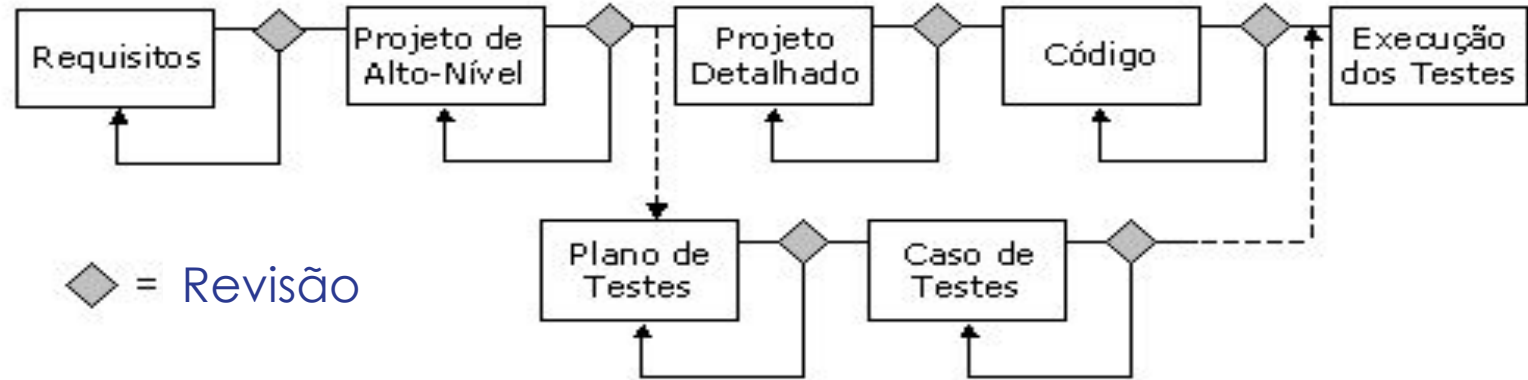
116 private boolean sendMail = true;
117 private Account.Id uploader;
118 private BatchRefUpdate batchRefUpdate;
119
120 @AssistedInject
121 public PatchSetInserter(ChangeHooks hooks,
122     ReviewDb db,

```

Stefan Beller Why do you move this out of the constructor? Initially I assumed this... Jan 28 2:55 PM
Dave Borowitz Because it would be identical between the two constructors, so it sa... Jan 28 3:19 PM

Revisões de Software

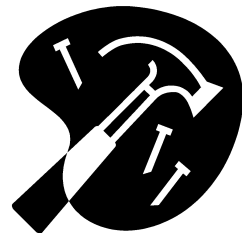
- Quando e em que tipos de artefato aplicar revisões de software?



Informação perdida; Informação transformada incorretamente;
Múltiplas transformações inconsistentes a partir da mesma fonte

Boas práticas de revisão de código

1. **Defina critérios claros** → Estabeleça padrões e aspectos a revisar (legibilidade, segurança, performance).
2. **Revise pequenos trechos** → Até 400 linhas para evitar fadiga e melhorar eficiência.
3. **Use ferramentas de revisão** → GitHub, GitLab, Bitbucket, linters e análise estática.
4. **Foque na qualidade** → Código limpo, modular e testável, evitando aprovações apressadas.
5. **Forneça feedback construtivo** → Seja objetivo, evite críticas pessoais e sugira melhorias.
6. **Automatize verificações** → Testes, linters e análise estática ajudam a reduzir erros manuais.
7. **Crie um ambiente colaborativo** → Incentive aprendizado, discussões e boas práticas.



O Processo de Revisão de Software

- Planejamento
 - Definir quem vai revisar o artefato, quando, as técnicas de detecção a serem empregadas:
 - Análise de código, compilação, revisão em pares, testes e simulação...
 - Revisões individuais ou em equipe.
 - Revisões em etapas.
- Preparação (opcional)
 - O autor do artefato pode descrever o artefato e a perspectiva utilizada em sua construção para o(s) revisor(es).
- Revisão Individual: Detecção de defeitos e Registro de defeitos

O Processo de Revisão de Software

- Reunião (opcional)

- Produzir uma lista de defeitos.
- Pesquisas recentes revelam que reuniões de revisão não contribuem significativamente para aumentar o número de defeitos encontrados.

- Correção

- Com base na lista de defeitos, o autor do artefato deve corrigi-lo ou explicar os motivos que levam alguns deles a não representarem problemas reais.

Revisões de Software

- Utilização na Prática
 - Muitas organizações realizam revisões de software.
 - Realização pouco sistematizada e potencial raramente é explorado.
 - No Brasil também tem sido muito utilizado (SBQS e MPS.BR)
- Tipos de Revisão de Software:
 - *Walkthroughs*.
 - Inspeções de Software.

Exercício Prático (10 min)

- Vamos aplicar uma atividade de revisão em requisitos
 - Em grupo de 3 pessoas, 5 minutos para escrever EM PAPEL regras de negócio relacionadas a 1 dos seguinte temas:
 - Matrícula de disciplinas no período
 - Atividades complementares
 - Monitoria
 - Acrescentar 1 ou 2 defeito(s) nas regras...
 - Trocar as regras entre as equipes...
 - Analisar as regras na tentativa de encontrar o(s) defeito(s) inserido(s).

Walkthroughs ... (na próxima semana)

- Reuniões informais para avaliação dos produtos
- Pouca ou nenhuma preparação requerida
- O desenvolvedor guia os presentes através do produto
- O objetivo é comunicar ou receber aprovação
- São úteis principalmente para requisitos e modelos.